Phase 1 of 6: Core language foundations · lesson 1 of 19

C++ Basics

Start here if C++ is new or rusty: how a program is built, variables and types, operators and control flow, functions and passing arguments, and the tools that catch mistakes. Pointers and References is the next lesson.

How a C++ program is built



Compile error: a problem inside one file. Linker error: declared and called but never defined.

Three stages. Know which one an error comes from.

C++ is **compiled** to machine code before it runs: fast, but it trusts you not to make memory mistakes.

main.cpp

```cpp
#include <iostream>          // paste in the declarations for std::cout

int main() {                 // execution starts here
    std::cout << "Hello, C++\n";
    return 0;                // 0 means success
}
```

1. **Preprocess**: `#include` literally pastes header files in; `#define` does text substitution.
2. **Compile**: each `.cpp` file becomes an *object file* on its own.
3. **Link**: the linker joins object files and libraries into one executable and resolves calls between files.

KEY POINT "Compile error vs linker error?" A compile error is a syntax or type problem inside one file. A linker error ("undefined reference") means a function was declared and called but never defined anywhere you linked.

Variables and types

Every variable has a **type** that fixes how much memory it uses and what you may do with it.

TYPE	HOLDS	TYPICAL SIZE
`bool`	`true` / `false`	1 byte
`char`	a character / small integer	1 byte (always, by definition)
`int`	whole number	4 bytes
`long long`	bigger whole number	8 bytes
`float` / `double`	decimal number	4 / 8 bytes

Only `char` has a fixed size. When size matters use `<cstdint>`: `std::int32_t`, `std::uint64_t`.

Initialization: always give a variable a value

```cpp
// BAD: indeterminate value, reading it is undefined behavior
int a;
int b = 5;       // copy initialization
int c(5);        // direct initialization
int d{5};        // brace (uniform) initialization: preferred
int e{};         // empty braces: zero-initialized, e == 0
int f{3.7};      // ERROR: braces forbid narrowing double → int
auto g = 5;      // g is int: the compiler deduces the type
const int h = 9;// cannot be changed after initialization
```

> UNDEFINED BEHAVIOR An uninitialized local holds garbage. Reading it is **UB**: junk, a crash, or "works" until it does not.

The signed/unsigned trap

`unsigned` wraps instead of going negative, and a mixed comparison converts the signed side to unsigned.

Operators and control flow

```
7 / 2      // 3     integer division truncates
7 % 2      // 1     remainder
// 3.5 one double operand makes the whole thing floating point
7 / 2.0

// && and || short-circuit: right side is skipped if not needed
if (x > 0 && y > 0) { }
for (int i = 0; i < 3; ++i) { }
for (int v : vec) { }     // range-based for: visits every element
while (cond) { }
// forget `break` and it falls through
switch (n) { case 1: /* ... */ break; default: break; }
```

> CLASSIC BUG `if (x = 5)` *assigns* 5 to x and is always true. You meant `==`. The compiler warns with `-Wall`.

Functions and passing arguments

Declare before use; the header holds declarations, the .cpp the definitions.

```
int add(int a, int b);                 // declaration
int add(int a, int b) { return a + b; }// definition
// overload: same name, different parameter types
double add(double a, double b);
void greet(std::string who = "world"); // default argument
```

Three ways to pass a parameter

```
void byValue(int n)        { n++; }          // works on a COPY
void byRef(int& n)         { n++; }          // works on the ORIGINAL
// original, read-only, no copy
void byConstRef(const std::string& s) { }
```

YOU WANT TO...	USE
read a small value (int, double, char, pointer)	by value: `int n`
read a large object without copying it	by const reference: `const std::string& s`
modify the caller's variable	by reference: `int& n`
accept "no object" as a valid argument	by pointer: `Widget* w`, and test it for `nullptr`
pass a run of elements	`std::span<int>` (C++20) or `const std::vector<int>&`, not a pointer plus a count
get a result back	return it by value; big objects such as `std::vector` are moved, not copied, so no out-parameter is needed

---

Tooling: warnings, sanitizers, static analysis

C++ trusts you, so let tools check what the language will not. Use them in layers, cheapest first.

LAYER	WHAT IT CATCHES	HOW
Compiler warnings	many beginner mistakes at once: uninitialized variables, signed/unsigned comparisons, missing returns	`-Wall -Wextra`, and `-Werror` in CI
Sanitizers	what only shows at run time: out-of-bounds, use after free, leaks, undefined behavior, data races	`-fsanitize=address,undefined` (and `-fsanitize=thread` for threads)
Static analyzer	rule violations, null dereferences, leaks and dead code found by reading the source, without running it	`clang-tidy`, `cppcheck`
Formatter	style, so reviews are about logic	`clang-format`

A CLEAN REPORT IS NOT A PROOF Analyzers give false positives and miss things, and a sanitizer only checks the paths your test actually ran. They reduce risk; they do not replace design, review and tests.

CODING STANDARDS Safety-critical projects (cars, medical devices) add written rules such as MISRA C++ or the AUTOSAR C++14 guidelines, checked automatically. The common themes are the ones in this course: initialize everything, no C-style casts, no naked `new` / `delete`, no silent narrowing, and no `using namespace` in headers.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
How a C++ program is built	C++ Primer	2.6 Defining our own data structures	101 to 109
	C++ Primer	3.1 Namespace using declarations	111 to 112
	C++ Primer	18.2 Namespaces	814 to 830
	Programming: Principles and Practice	7.2 to 7.3 Declarations and definitions; scope	194 to 202
Variables and types	Programming: Principles and Practice	2.5 Assignment and initialization	49 to 52
	Programming: Principles and Practice	2.8 to 2.10 Type safety, conversions, type deduction with auto	56 to 60
	Effective Modern C++	Item 7, Distinguish () and {} when creating objects	67 to 76
Operators and control flow	C++ Primer	4.2 to 4.3 Arithmetic, logical and relational operators	168 to 172
	C++ Primer	4.4 to 4.5 Assignment, increment and decrement	173 to 178
	C++ Primer	5.3 to 5.5 Conditional, iterative and jump statements	203 to 221
Functions and passing arguments	C++ Primer	6.1 Function basics	231 to 236
	C++ Primer	6.2 Argument passing	237 to 250
	C++ Primer	6.3 Return types and the return statement	251 to 258
	C++ Primer	6.4 Overloaded functions	259 to 264
	C++ Primer	6.6 Function matching	271 to 275
	Programming: Principles and Practice	3.5 to 3.6 Functions; vector introduced	81 to 89
	Programming: Principles and Practice	7.2 to 7.3 Declarations and definitions; scope	194 to 202
	Programming: Principles and Practice	7.4 Function call and return (pass by value, by reference, by const reference)	203 to 218